

A Comparative Analysis of Monolithic and Microservices Architectures: Evaluating Fault Isolation and Database Resource Contention Through Experimental Stress Testing

Ahmad Louay, Bahaa El Deen Mohamed, Youssef Khaled
Sadat Academy for Management Sciences

1. Abstract

Microservices are widely adopted in large systems for organizational scalability and service-level autonomy. However, this architectural style can introduce data-tier and network-level costs that are not always visible in high-level design discussions. In this work, we compare monolithic and microservices implementations of the same e-commerce workload under controlled, repeatable experiments. Both systems share equivalent business logic, identical datasets, and comparable application-level compute budgets. We evaluate four scenario families: baseline load, deterministic endpoint failure, fixed latency injection, and database connection pool exhaustion. Measurements are collected using Apache JMeter and analyzed using per-run and cross-scenario statistics, including means, confidence intervals, architecture deltas, and degradation relative to baseline. Results show that the monolith delivers substantially higher baseline throughput and lower tail latency in our environment, while microservices exhibit greater sensitivity to downstream degradation in order-processing paths. Under latency injection, both architectures degrade sharply, but microservices show stronger tail-latency inflation and higher error rates. Pool-size sweeps reveal non-linear contention behavior and architecture-dependent tuning effects, reinforcing the need for explicit data-tier capacity planning.

2. Introduction

The term “monolith” implies something large and tightly integrated, which reflects a key practical advantage of monolithic software design: one codebase usually simplifies deployment, tracing, and local testing, and often delivers strong baseline performance in many environments [1], [2]. These properties make monolithic architecture a practical choice for early-stage products, where development speed, lower operational complexity, and lower platform overhead are important. In contrast, moving to microservices can be complex and costly and usually requires stronger domain boundaries and operational maturity [3], [4]. However, monolithic systems can become harder to evolve at large scale. Tight coupling can slow technology adoption and independent scaling, and major changes may require broad system-level modification rather than service-level evolution [5], [6].

Because of these long-term limitations, many organizations shift toward microservices, which decompose the system into smaller independently deployable services with clearer responsibility boundaries [5], [7].

However, the decentralization inherent in microservices does not come without operational costs. While functional logic is easily partitioned, the data tier often remains a point of contention. In a monolithic system, database interactions are managed through a unified connection pool, optimizing resource utilization. Conversely, a microservices deployment requires each independent service to maintain its own connection overhead. As the number of services scales, this leads to ‘connection sprawl,’ where the aggregate demand for database handles can exhaust system resources even if the underlying hardware is underutilized.

Furthermore, while microservices are designed to prevent cascading failures—ensuring that a bug in one module does not collapse the entire ecosystem—the inter-service communication overhead introduces new latency patterns. Under peak load, the P95 latency (the time within which 95% of requests are completed) can degrade significantly due to network hops and resource competition, a phenomenon that is often less pronounced in the shared-memory environment of a monolith.

This study quantifies these trade-offs using a controlled benchmark campaign across monolithic and microservices implementations of the same domain. This paper focuses on latency, throughput, error behavior, and connection-pool stress to provide evidence-based guidance on when architectural decomposition introduces measurable infrastructure cost.

3. Methodology

3.1. Study Design

This study follows a controlled comparative design to isolate architectural effects under equivalent workload conditions. Experimental interpretation follows standard performance-analysis practice, emphasizing repeatability, controlled variables, and statistical summaries [8].

3.2. Systems Under Test

Both systems implement the same domain entities (users, products, and orders) and the same business rules for read and order-creation paths.

- The monolith is a single Spring Boot service [9] backed by one relational database and one application connection pool [10].
- The microservices system includes user, product, and order services behind an API gateway [11]; each service owns its database and connection pool.
- Inter-service communication in the microservices variant uses synchronous HTTP principles aligned with REST architecture [12].

To preserve fairness, both deployments use the same dataset and comparable total CPU and memory limits at the application tier.

3.3. Workload Model

Load is generated with Apache JMeter [13] using a read-heavy profile intended to represent common catalog traffic.

Operation class	Share
Product reads	50%
User reads	30%
Order creation	20%

Each run includes ramp-up, warm-up, and steady-state phases. Warm-up samples are excluded from all reported metrics.

3.4. Experimental Scenarios

We evaluate four scenario families.

Scenario	Intent	Configuration summary
Baseline	Reference	No injected faults or delays; default pool settings

Scenario	Intent	Configuration summary
Deterministic endpoint failure	Failure propagation	Product-read failures injected for a fixed ID set
Latency injection	Performance degradation	Fixed delay injected on product-read path
Pool exhaustion sweep	Data-tier contention	Connection pool sizes swept across 2, 5, 10 with fixed request load

Fault and latency injection are implemented at the same logical point in both architectures (product-read path) and controlled by runtime properties: `chaos.enabled`, `chaos.mode`, `chaos.fault-ids`, and `chaos.latency-ms` [14].

For deterministic failure runs, a request fails if and only if its product ID belongs to a configured fixed set. This avoids random sampling noise and improves reproducibility across repeated runs and across architectures.

3.5. Connection Pool Stress Protocol

For pool stress tests, maximum pool size is varied while request concurrency is held constant. This isolates database-handle scarcity from other factors.

- Pool sizes tested: 2, 5, 10
- Per-size repeated runs are executed and analyzed independently for both architectures
- Per-run environment is reset to avoid state carry-over
- Pool and connection limits are interpreted with reference to pool configuration behavior and database connection ceilings [10], [15]

3.6. Metrics and Statistical Treatment

We focus on the following outcome variables:

- Throughput (requests/second)
- Latency percentiles (P50, P95, P99)
- Error rate and error count
- Time-series behavior for throughput and tail latency

For each scenario, results are reported at run level and as architecture-level aggregates. The analysis pipeline computes means, standard deviations, and 95% confidence intervals, then reports architecture deltas and degradation relative to baseline where baseline data is available.

3.7. Execution and Reproducibility

Each run is executed in isolation with Docker Compose [16].

1. Start architecture and wait for health checks
2. Execute the JMeter plan
3. Persist raw `.jtl` output and HTML report
4. Tear down containers and volumes
5. Apply cooldown before the next run

Outputs are organized by timestamp and scenario for traceability. Scenario metadata is persisted for every run, and both raw measurements and processed summaries are retained at scenario and experiment-aggregate levels. Reproducibility practices follow the same rationale used in computational research workflows [17].

4. Experimental Results

This section reports the final campaign at 300 concurrent threads with three repeated runs per scenario. Metrics are presented as architecture-level means with 95% confidence intervals (CI).

Cross-scenario aggregates provide a compact view of architecture-level differences across baseline, failure, latency, and pool-stress conditions.

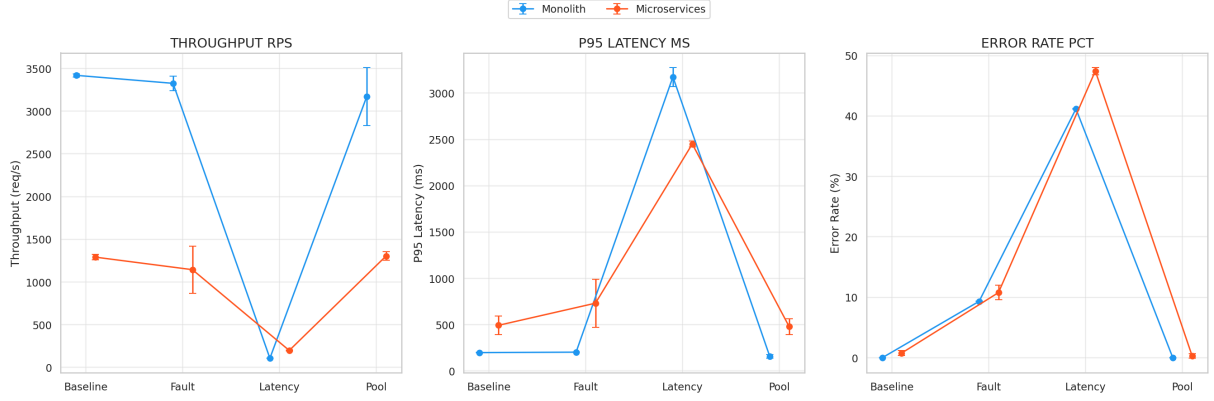


Figure 1: Cross-scenario comparison of throughput, P95 latency, and error rate (mean with 95% confidence intervals).

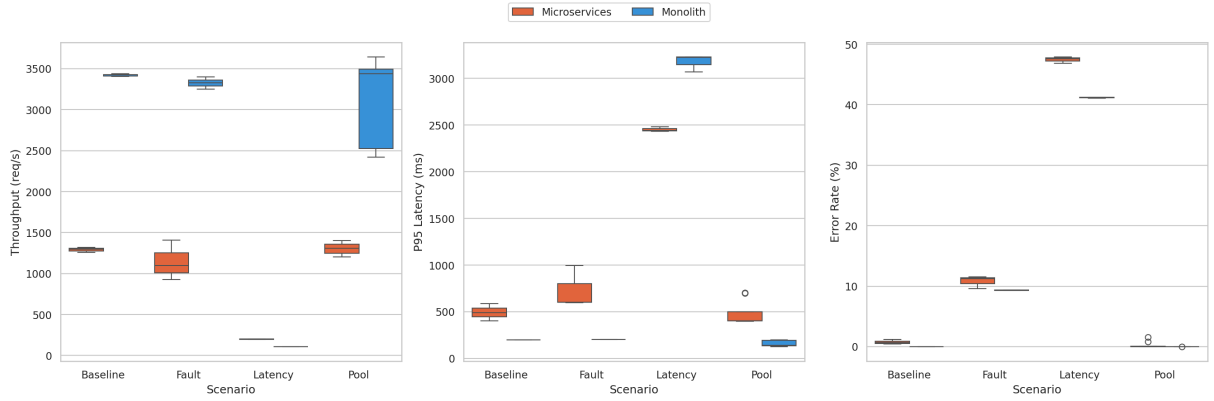


Figure 2: Run-level metric distributions by scenario and architecture.

4.1. Baseline Performance

Under baseline load, the monolith achieved substantially higher throughput and lower tail latency than the microservices deployment.

- Monolith mean throughput: 3420.1 req/s (CI +- 18.9)
- Microservices mean throughput: 1291.1 req/s (CI +- 33.4)
- Throughput delta (micro vs mono): - 62.25%
- Monolith mean P95: 199.0 ms (CI +- 1.13)
- Microservices mean P95: 493.7 ms (CI +- 102.4)

Baseline error behavior also differs. Monolith remained effectively error-free at aggregate level, while microservices showed non-zero baseline errors concentrated in POST /orders, indicating that the composite order path is the dominant fragility point under high concurrency.

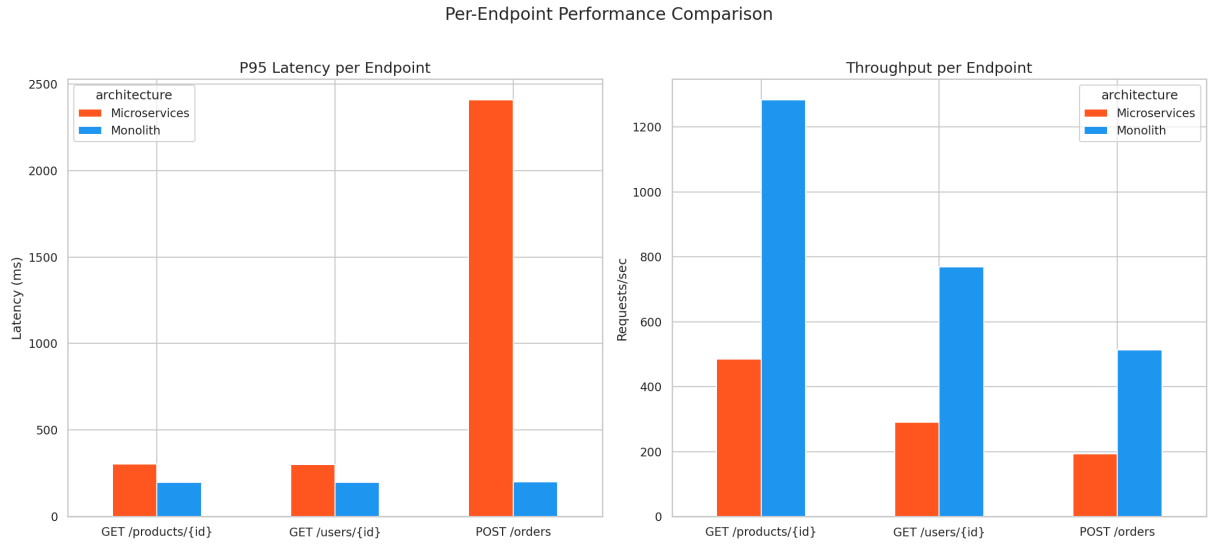


Figure 3: Baseline endpoint comparison between architectures (throughput and latency characteristics by API path).

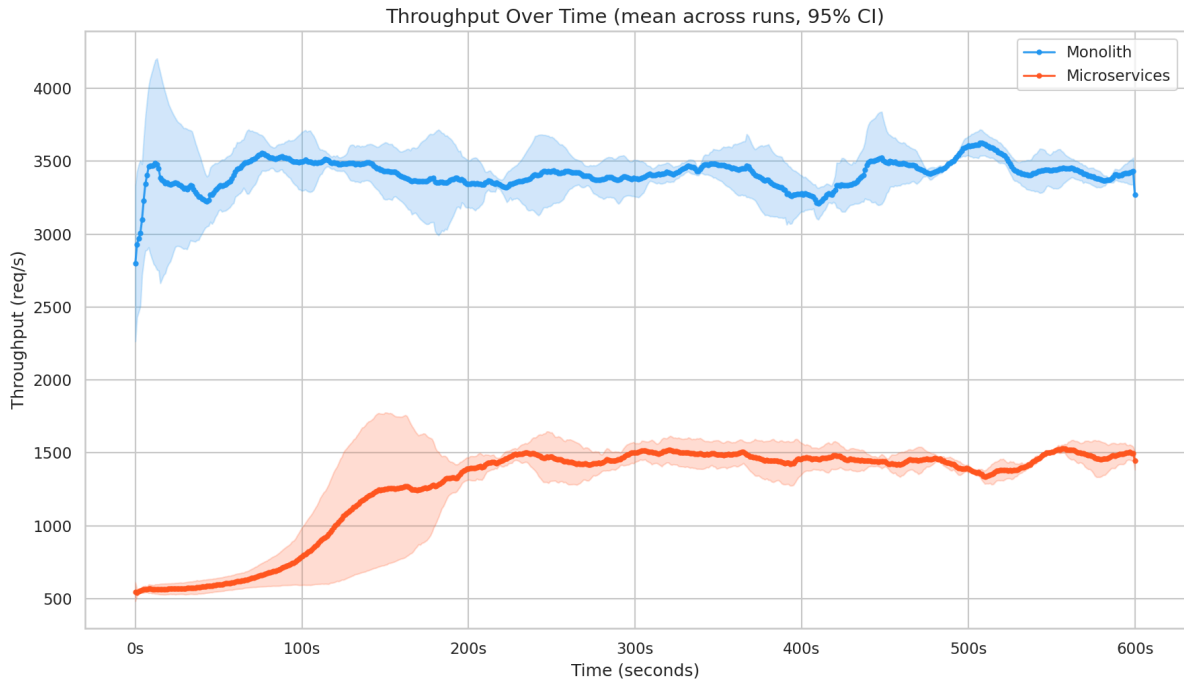


Figure 4: Baseline throughput over normalized run time (mean across runs).

4.2. Deterministic Endpoint Failure

Deterministic fault injection used the fixed ID set 3, 7, 11, 12, which represents $4/30 = 13.33\%$ of product IDs. Observed product-read error rates match this expected fault share in both architectures.

- GET /products/{id} error rate: 13.36% (microservices) vs 13.33% (monolith)
- Scenario-level error rate: 10.78% (microservices) vs 9.33% (monolith)
- Throughput: 1144.6 req/s (microservices) vs 3326.4 req/s (monolith)
- P95 latency: 732.3 ms (microservices) vs 203.7 ms (monolith)

Endpoint-level decomposition shows stronger downstream amplification in microservices order creation.

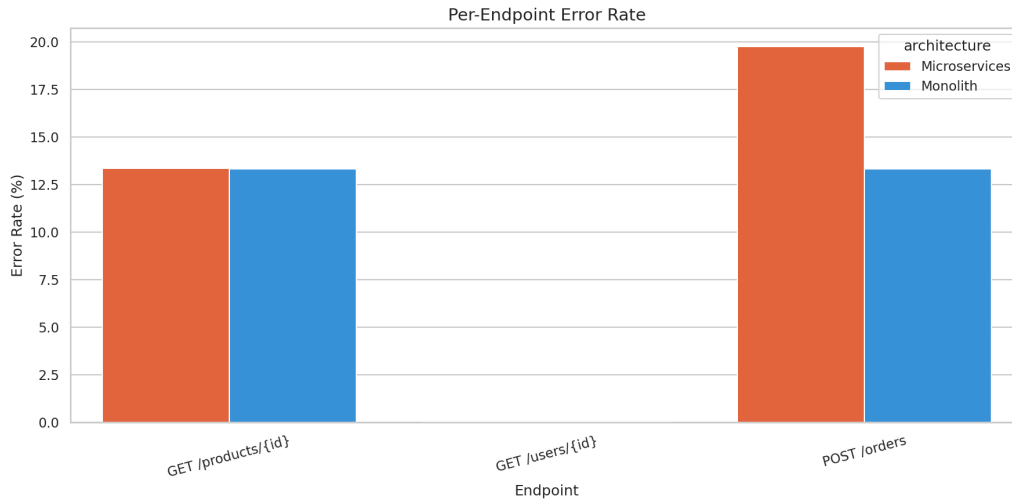


Figure 5: Per-endpoint error rates during deterministic endpoint failure runs.

4.3. Latency Injection

Latency injection produced the strongest degradation in both architectures, with severe tail-latency growth and error-rate increases.

- Monolith: 107.5 req/s, P95 3175.0 ms, 41.18% errors
- Microservices: 198.7 req/s, P95 2450.9 ms, 47.44% errors

Compared with baseline, both architectures show major throughput collapse and tail-latency inflation. Endpoint decomposition also reveals architecture-specific coupling: GET /users/{id} in microservices remains near-zero error under product-path latency, while POST /orders becomes the dominant failure path in both systems.

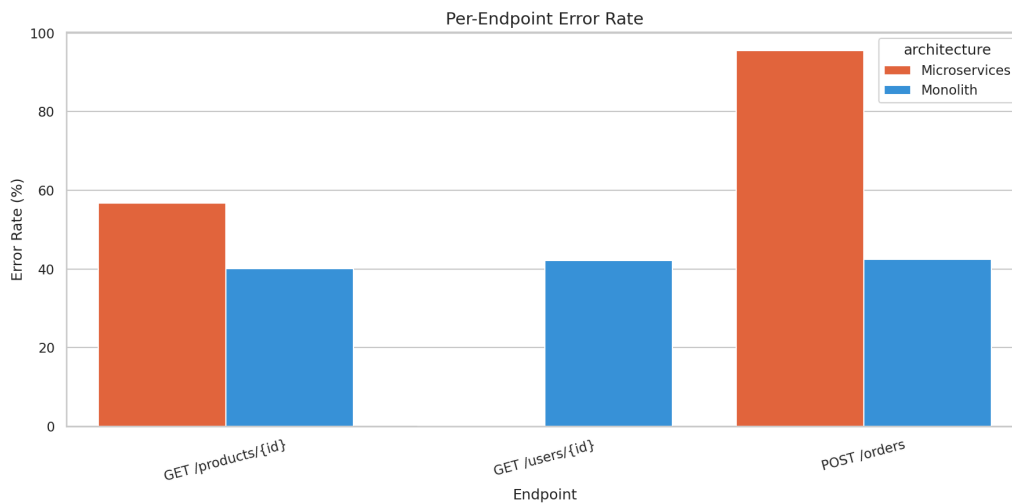


Figure 6: Per-endpoint error rates under latency injection.

4.4. Pool Exhaustion Sweep

Pool-size sweeps reveal architecture-dependent contention behavior.

- Monolith throughput changes strongly with pool size: 2482.6 -> 3514.2 -> 3518.9 req/s for sizes 2, 5, 10
- Monolith P95 remains low at 2/5 and increases at 10: 138.3, 137.7, 197.3 ms
- Microservices throughput varies within a narrower band: 1382.1, 1243.8, 1289.6 req/s
- Microservices error remains low at 2/5 but rises at 10: 0.0035%, 0.0013%, 0.8232%

This non-linear pattern suggests that connection-pool tuning should be architecture-specific rather than transferred directly between deployment styles.

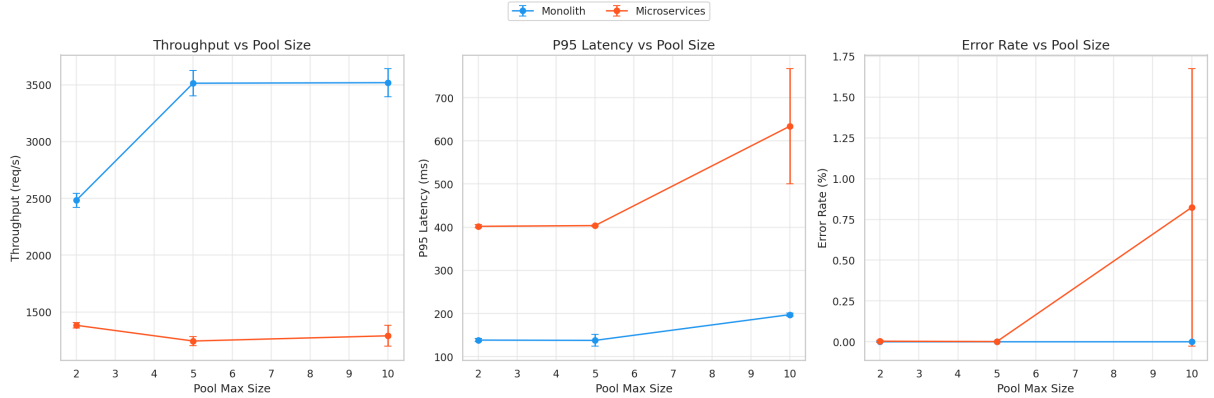


Figure 7: Pool-size sweep: throughput, P95 latency, and error rate versus maximum pool size.

5. Discussion

Four findings emerge from these results.

First, under this workload and compute budget, monolith retains a clear baseline efficiency advantage in both throughput and tail latency. The magnitude of this gap indicates lower orchestration overhead and lower cross-service tail amplification in the monolithic path [18], [19].

Second, deterministic fault injection yields product-read error rates that match the configured fault-ID share across architectures, strengthening internal validity. Both architectures exhibit propagation into composite operations, but microservices show stronger downstream amplification in POST /orders [20].

Third, latency injection exposes different coupling characteristics. Microservices preserve near-zero error on unrelated GET /users/{id} despite severe product-path degradation, while both architectures experience major failure in order creation. This indicates partial isolation benefits in microservices, but also substantial end-to-end fragility in composition-heavy paths [20].

Fourth, pool-stress behavior is non-monotonic and architecture-specific. Monolith is highly sensitive to aggressive under-provisioning (pool=2) and stabilizes at higher pool sizes; microservices show weaker throughput response but can incur higher tail latency and error emergence at larger pools. This confirms that pool policies should be tuned per architecture and endpoint criticality rather than copied across deployment styles [10], [15].

From an engineering perspective, both architectures require explicit SLO-oriented controls under stress (timeouts, bounded retries, backpressure, and connection governance). For microservices, resilience controls on composite order paths are especially important to prevent localized faults from becoming user-visible failures [20], [21].

5.1. Threats to Validity

- External validity: findings reflect one domain model, one workload mix, and one implementation stack.
- Internal validity: deterministic injection improves reproducibility but does not represent random production-failure distributions.
- Construct validity: endpoint-level fault injection evaluates propagation behavior, not orchestrator-driven restart resilience.

- Resource validity: CPU and memory are balanced at the application tier, but architecture-intrinsic topology differences (single database vs service-owned databases) remain part of the treatment.

5.2. Reproducibility Artifact

Source code and experiment automation are available at:
the project repository [22].

6. Conclusion

In this benchmark, the monolith delivered higher baseline throughput and lower tail latency, and it was less sensitive to injected downstream degradation. The microservices variant exposed stronger propagation and tail effects under stress, alongside architecture-dependent data-tier tuning behavior.

These results do not imply a universal winner. Microservices still offer organizational advantages, but they require stronger operational discipline and explicit resource governance to achieve stable runtime behavior. Architecture decisions should therefore be based on workload profile, team structure, reliability targets, and operational maturity rather than trend adoption.

7. Appendix

Additional diagnostic visualizations are provided for transparency and replication support.

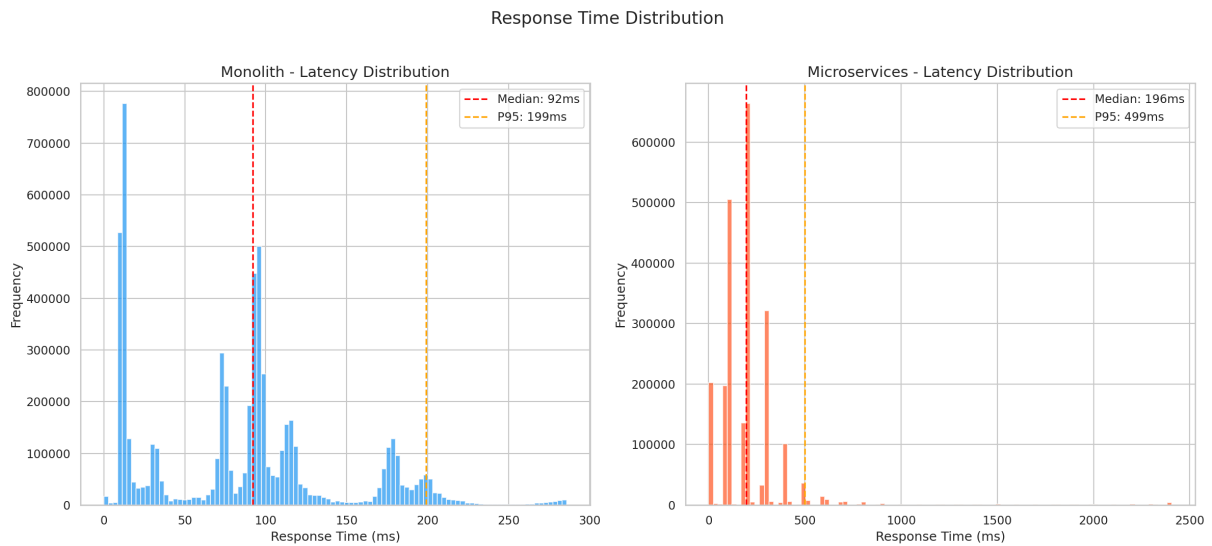


Figure 8: Baseline latency distribution by architecture.

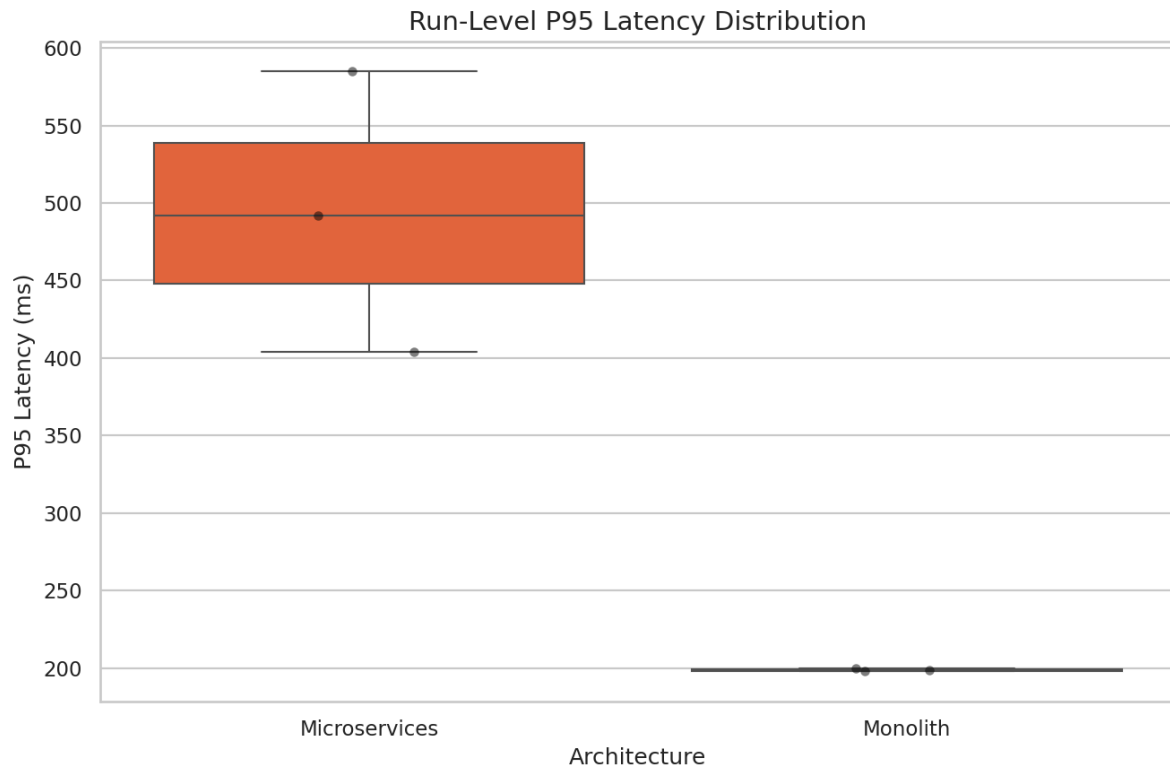


Figure 9: Baseline run-level P95 latency variability across repeated runs.

References

- [1] Atlassian, “Microservices vs. Monolith Architecture.” [Online]. Available: <https://www.atlassian.com/microservices/microservices-architecture/microservices-vs-monolith>
- [2] M. Fowler, “Monolith First.” [Online]. Available: <https://martinfowler.com/bliki/MonolithFirst.html>
- [3] D. Taibi, V. Lenarduzzi, and C. Pahl, “Processes, motives, and challenges for migrating from monolithic to microservices architecture: An industrial survey,” *Software Quality Journal*, vol. 25, no. 3, pp. 743–770, 2017.
- [4] P. Jamshidi, C. Pahl, N. C. Mendonça, J. Lewis, and S. Tilkov, “Microservices: The journey so far and challenges ahead,” *IEEE Software*, vol. 35, no. 3, pp. 24–35, 2018.
- [5] S. Newman, *Building Microservices*, 2nd ed. O'Reilly Media, 2021.
- [6] N. Dragoni *et al.*, “Microservices: Yesterday, Today, and Tomorrow,” *Present and Ulterior Software Engineering*. Springer International Publishing, pp. 195–216, 2017. doi: 10.1007/978-3-319-67425-4_12.
- [7] C. Richardson, *Microservices Patterns: With Examples in Java*. Manning Publications, 2018.
- [8] R. Jain, *The Art of Computer Systems Performance Analysis: Techniques for Experimental Design, Measurement, Simulation, and Modeling*. John Wiley, Sons, 1991.
- [9] Spring Team, “Spring Boot.” [Online]. Available: <https://spring.io/projects/spring-boot>
- [10] Brett Wooldridge and Contributors, “HikariCP.” [Online]. Available: <https://github.com/brettwooldridge/HikariCP>

- [11] C. Richardson, “Building Microservices: Using an API Gateway.” [Online]. Available: <https://www.f5.com/company/blog/nginx/building-microservices-using-an-api-gateway>
- [12] R. T. Fielding, “Architectural Styles and the Design of Network-based Software Architectures,” Doctoral dissertation, 2000. [Online]. Available: <https://roy.gbiv.com/pubs/dissertation/top.htm>
- [13] Apache Software Foundation, “Apache JMeter.” [Online]. Available: <https://jmeter.apache.org/>
- [14] Chaos Engineering Community, “Principles of Chaos Engineering.” [Online]. Available: <https://principlesofchaos.org/>
- [15] PostgreSQL Global Development Group, “PostgreSQL Documentation: Connections and Authentication.” [Online]. Available: <https://www.postgresql.org/docs/current/runtime-config-connection.html>
- [16] Docker, Inc., “Docker Compose.” [Online]. Available: <https://docs.docker.com/compose/>
- [17] R. D. Peng, “Reproducible Research in Computational Science,” *Science*, vol. 334, no. 6060, pp. 1226–1227, 2011, doi: 10.1126/science.1213847.
- [18] J. Dean and L. A. Barroso, “The Tail at Scale,” *Communications of the ACM*, vol. 56, no. 2, pp. 74–80, 2013, doi: 10.1145/2408776.2408794.
- [19] Y. Gan *et al.*, “An Open-Source Benchmark Suite for Microservices and Their Hardware-Software Implications for Cloud and Edge Systems,” in *Proceedings of the Twenty-Fourth International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2019, pp. 3–18. doi: 10.1145/3297858.3304013.
- [20] B. Beyer, C. Jones, J. Petoff, and N. R. Murphy, *Site Reliability Engineering: How Google Runs Production Systems*. O'Reilly Media, 2016.
- [21] M. T. Nygard, *Release It!: Design and Deploy Production-Ready Software*, 2nd ed. The Pragmatic Programmers, 2018.
- [22] Bahaio, “Monolith vs Microservices Benchmark Repository.” [Online]. Available: <https://github.com/Bahaio/monolith-vs-microservices-benchmark>